

```

1 begin
2   using Oscar: ZZ, QQ, matrix_space
3   using Latexify
4 end

```

[1//3;1//3;1//3]

```

1 begin
2   # Global Mathematical Domain Bindings
3   Z = ZZ
4   Q = QQ
5   Num_Nodes = Z(3)
6
7   Space_Z = matrix_space(Z, 3, 3)
8   Space_Q = matrix_space(Q, 3, 3)
9
10  # Adjacency Matrix representing direct walks
11  Adj_Alg = Space_Z([
12    0 1 1;
13    1 0 1;
14    1 0 0
15  ])
16
17  # Transition Matrix for Markov Chain (probabilities as rationals)
18  M_Alg = Space_Q([
19    0 Q(1,2) Q(1,2);
20    Q(1,2) 0 Q(1,2);
21    1 0 0
22  ])
23
24  # Initial State Vector (Vector Space over Q)
25  V_Q = matrix_space(Q, 3, 1)
26  s_0 = V_Q([Q(1,3), Q(1,3), Q(1,3)])
27 end

```

Lecture Seventeen: Ways to Walk— Matrices and Markov Chains

Scope

- Focuses on **combinatorics of paths** and **Markov chains**.
- Explores how multiplying adjacency matrices reveals the number of walks in a graph.
- Introduces Markov chains for analyzing systems that change randomly over time.

I. Matrices as walks

A. The number of k -step walks from vertex i to vertex j is the (i, j) entry of A^k . B. Problem: Find the number of 3-step walks between all nodes.

3×3 Matrix{Int64}:

```
1 2 2
2 1 2
2 0 1
```

```
begin
  # SECTION I - PARADIGMS
  # Calculate 3-step walks

  # 1. Pure FP
  walks_fp = let
    n = Int(Num_Nodes)
    a_raw = [Int(Adj_Alg[i,j]) for i in 1:n, j in 1:n]

    # Matrix multiplication recursively
    mat_mul = (X, Y) -> [sum(X[i,k]*Y[k,j] for k in 1:n) for i in 1:n, j in 1:n]

    a2 = mat_mul(a_raw, a_raw)
    mat_mul(a2, a_raw)
  end

  # 2. Abstract Algebra
  walks_aa = let
    A3 = Adj_Alg^3
    [Int(A3[i,j]) for i in 1:3, j in 1:3]
  end

  # 3. Geometric Focus
  walks_grp = let
    A3 = Adj_Alg * Adj_Alg * Adj_Alg
    [Int(A3[i,j]) for i in 1:3, j in 1:3]
  end
end
```

Aggregate Verification (Section I):

- **Pure FP:** [1 2 2; 2 1 2; 2 0 1]
- **Abstract Algebra:** [1 2 2; 2 1 2; 2 0 1]
- **Geometric:** [1 2 2; 2 1 2; 2 0 1]

All paradigms agree: **true**

```
begin
  Markdown.parse("""
    ### Aggregate Verification (Section I):
    * **Pure FP**: $(walks_fp)
    * **Abstract Algebra**: $(walks_aa)
    * **Geometric**: $(walks_grp)

    All paradigms agree: **$(walks_fp == walks_aa == walks_grp)**
    """)
end
```

II. Markov Chains

A. A Markov chain describes a sequence of possible events where the probability of each event depends only on the state attained in the previous event. B. Problem: Find the state vector after 2 transitions.

[5//12, 1//4, 1//3]

```
begin
  # SECTION II - PARADIGMS
  # Calculate the state after 2 steps ( $M^2 * s_0$  for left-multiplication, but here
  # rows are states?
  # Wait, usually it is  $s_0^T * M^2$ . Let's compute  $s_2$ ).

  # 1. Pure FP
  state_2_fp = let
    n = Int(Num_Nodes)
    m_raw = [M_Alg[i,j] for i in 1:n, j in 1:n]
    s_raw = [s_0[i,1] for i in 1:n]

    #  $s_1 = M^T * s_0$ 
    s1 = [sum(m_raw[j,i] * s_raw[j] for j in 1:n) for i in 1:n]
    s2 = [sum(m_raw[j,i] * s1[j] for j in 1:n) for i in 1:n]
    s2
  end

  # 2. Abstract Algebra
  state_2_aa = let
    M2 = M_Alg^2
    s2_mat = transpose(M2) * s_0
    [s2_mat[i,1] for i in 1:3]
  end

  # 3. Geometric Focus
  state_2_grp = let
    s1_mat = transpose(M_Alg) * s_0
    s2_mat = transpose(M_Alg) * s1_mat
    [s2_mat[i,1] for i in 1:3]
  end
end
```

Aggregate Verification (Section II):

- **Pure FP State 2:** $\text{Nemo.QQFieldElem}[5//12, 1//4, 1//3]$
- **Abstract Algebra State 2:** $\text{Nemo.QQFieldElem}[5//12, 1//4, 1//3]$
- **Geometric State 2:** $\text{Nemo.QQFieldElem}[5//12, 1//4, 1//3]$

All paradigms agree: **true**

```
begin
  Markdown.parse("""
    ### Aggregate Verification (Section II):
    * **Pure FP State 2**:  $\text{Nemo.QQFieldElem}[5//12, 1//4, 1//3]$ 
    * **Abstract Algebra State 2**:  $\text{Nemo.QQFieldElem}[5//12, 1//4, 1//3]$ 
    * **Geometric State 2**:  $\text{Nemo.QQFieldElem}[5//12, 1//4, 1//3]$ 

    All paradigms agree:  $\text{Nemo.QQFieldElem}[5//12, 1//4, 1//3]$ 
    """)
end
```

III. The Stable Matrix

A. A regular Markov chain eventually reaches a stable state (equilibrium) independent of the initial state. B. Problem: Find the limit matrix $\mathbf{M}^\infty$ representing the steady state. For this simple 3-node graph, we will compute $\mathbf{M}^4$ as a proxy for approaching stability.

3x3 Matrix{Nemo.QQFieldElem}:

```
9//16  1//8   5//16
1//2   3//16  5//16
1//4   3//8   3//8
```

```
begin
  # SECTION III - PARADIGMS
  # Calculate M^4

  # 1. Pure FP
  m4_fp = let
    n = Int(Num_Nodes)
    m_raw = [M_Alg[i,j] for i in 1:n, j in 1:n]
    mat_mul = (X, Y) -> [sum(X[i,k]*Y[k,j] for k in 1:n) for i in 1:n, j in 1:n]
    m2 = mat_mul(m_raw, m_raw)
    mat_mul(m2, m2)
  end

  # 2. Abstract Algebra
  m4_aa = let
    M4 = M_Alg^4
    [M4[i,j] for i in 1:3, j in 1:3]
  end

  # 3. Geometric Focus
  m4_grp = let
    M2 = M_Alg * M_Alg
    M4 = M2 * M2
    [M4[i,j] for i in 1:3, j in 1:3]
  end
end
```

Aggregate Verification (Section III):

- **Pure FP:** Nemo.QQFieldElem[9//16 1//8 5//16; 1//2 3//16 5//16; 1//4 3//8 3//8]
- **Abstract Algebra:** Nemo.QQFieldElem[9//16 1//8 5//16; 1//2 3//16 5//16; 1//4 3//8 3//8]
- **Geometric:** Nemo.QQFieldElem[9//16 1//8 5//16; 1//2 3//16 5//16; 1//4 3//8 3//8]

All paradigms agree: **true**

```
begin
  Markdown.parse("""
    ### Aggregate Verification (Section III):
    * **Pure FP**: $(m4_fp)
    * **Abstract Algebra**: $(m4_aa)
    * **Geometric**: $(m4_grp)

    All paradigms agree: **$(m4_fp == m4_aa == m4_grp)**
    """)
end
```